

Formal Entity-Relationship Knowledge Graph

System Architecture, Execution Guide, Modifications & Future Roadmap

Executive Summary: This document provides an exhaustive reference manual for the newly implemented formal Knowledge Graph (KG) engine powering the campus chatbot and web portal. The system models academic departments, multi-tier curriculum syllabi, faculty assignments, and campus physical infrastructure (buildings, floors, rooms, and labs). It replaces heuristic/pseudo-random graphs with a deterministic, ontology-validated directed relational network containing **422 nodes** across **22 entity types**.

1. System Setup & Step-by-Step Execution Guide

Follow these instructions to run the web server, launch the assistant, and verify graph integrity:

Step 1: Environment Activation & Dependency Verification

Open terminal (PowerShell or Bash) in the project root directory and ensure packages are installed:

```
pip install -r requirements.txt
```

Step 2: Automated Test Suite Verification

Execute the dedicated 62-point verification suite to ensure all graph entities, relationships, queries, and NLP intents function properly:

```
python test_er_kg.py
```

Step 3: Launching the Flask Web Server

Start the server locally on port 5000:

```
python app.py
```

Upon execution, the terminal will report default administrative credentials if it is the first run, and listen on `http://localhost:5000`.

Step 4: Accessing Application Endpoints

Endpoint URL	Function & Target View
http://localhost:5000/	Main Campus Web Portal & Integrated AI Assistant
http://localhost:5000/er-diagram	Interactive Formal Entity-Relationship (ER) Schema Visualizer
http://localhost:5000/api/er-schema	Raw Mermaid-formatted ER schema export endpoint
http://localhost:5000/api/er-summary	JSON metadata endpoint of entity & relationship specs
http://localhost:5000/api/graph-stats	Real-time Node and Edge breakdown counts by category
http://localhost:5000/admin	Admin Management Console (Auth required)

2. Comprehensive Audit of Modifications Made

The project underwent systematic refactoring to transition from a shallow structure into a formal Entity-Relationship Knowledge Graph:

Module / File	Nature of Modification	Technical Impact
data/college_data.json	Data Expansion	Added campus_infrastructure (11 buildings, 37 rooms, labs), structured curriculum (116 subjects across 8 depts), and deterministic faculty assignments.
knowledge_graph/ontology.py	Schema Definition	Introduced 5 new entity types (Building, Floor, Room, Semester, Program) and 9 formal relationship types (has_building, has_room, housed_in, etc.).
knowledge_graph/er_schema.py	New Subsystem	Provides export_er_diagram() (Mermaid), export_er_summary() (metadata API), and validate_graph_against_schema() automated validator.
knowledge_graph/kg_builder.py	Full Engine Rewrite	Removed all pseudo-random hash()%N links. Replaced with modular, deterministic graph building pipelines.
knowledge_graph/kg_query.py	Query Extension	Added deep traversal methods: query_building(), query_campus_infrastructure(), query_faculty_profile(), and query_subjects_by_semester().
knowledge_graph/nlp_processor.py	NLP Enhancement	Added 3 new intents (building_info, faculty_profile, subject_info) with regex entity extractors for faculty names and building codes.
templates/er_diagram.html	Visualization UI	Interactive Mermaid.js schema diagram with stat summary cards and responsive green-slate aesthetic.
app.py	Server Routes	Mounted /er-diagram, /api/er-schema, /api/er-summary and wired new query handlers into chat pipeline.

test_er_kg.py	Validation Suite	62-test verification suite asserting graph counts, assignments, curriculum integrity, and query routing.
---------------	------------------	--

3. Technology Stack & Frameworks Utilized

The platform is engineered using modern Python and Web standards for maximum speed and portability:

Layer / Component	Technology / Library	Role in System
Backend Web Framework	Flask 3.1.1 & Werkzeug	Application server, REST API routing, and Jinja2 templating.
Graph Engine	NetworkX 3.4.2	Directed in-memory Knowledge Graph representation and multi-hop graph traversal.
NLP & Matching	scikit-learn (TF-IDF) & Re	Vector space intent classification with cosine similarity and regex entity extraction.
ORM & Database	SQLAlchemy & SQLite	Relational persistence for authentication, user sessions, and chat conversation logging.
Security & Sessions	Flask-Login & Werkzeug Security	Role-based access control, session state management, and password hashing.
Visualization	Mermaid.js & D3.js	Client-side declarative ER diagram rendering and dynamic force-directed network graphs.
Document Generation	ReportLab 4.5.1	Programmatic compilation of formal PDF documentation and reports.

4. Strategic Roadmap & Future Enhancements

To elevate the platform from a rule/graph-augmented assistant into an enterprise campus AI, the following enhancements are recommended:

- **Phase 1: Neo4j / Graph Database Persistence**
Migrate the in-memory NetworkX graph to an external Cypher-compatible database (Neo4j or Amazon Neptune). Enables real-time concurrent graph edits, graph clustering algorithms, and sub-millisecond querying over millions of campus nodes.
- **Phase 2: Hybrid Graph-RAG with Local LLM (Ollama / Llama-3)**
Integrate small quantized LLMs (e.g. Llama-3-8B via Ollama). Use the Knowledge Graph to extract verified facts and inject them into LLM prompt contexts, eliminating hallucination while providing fluent conversational answers.
- **Phase 3: Real-Time Dynamic Timetables & Smart Campus IoT**
Expand Room and Faculty nodes to connect with live timetable scheduling and IoT occupancy sensors. Students can ask: 'Is Lab 201 free right now?' or 'Where is Dr. Ramarao currently lecturing?'
- **Phase 4: Semantic Vector Embeddings on Entities**

Store dense embeddings (Sentence-Transformers) for each node and subject description, enabling multi-lingual query understanding and semantic fuzzy entity disambiguation.

- **Phase 5: Student Role Self-Service Portal**

Enable authenticated students to query personal degree audits, check prerequisites, and track course credits directly against the curriculum graph.

Report generated automatically by Antigravity Agent. Sphoorthy Engineering College Campus Assistant.